



Repàs exprés · Ràdio

Per a qui és? Per a l'**alumnat**, com a **deures de final de curs** (es reparteix a la SA9, abans de la prova pràctica T3), juntament amb `00_Repas_expres_MicroPython.md`. Mateixa mecànica: **escriu primer de memòria**, comprova després amb la solució. Si falles, repassa la Part B (ràdio) de `SA0_guia_programacio.md` i torna-hi l'endemà.

Com s'usa: 10-15 minuts per tanda. Per provar-ho de debò calen dues plaques (com el comandament i el vehicle de la SA5), però per repassar la sintaxi n'hi ha prou amb el quadern.

R1 · El ritual de la ràdio

Tres línies s'han d'escriure **sempre, abans del bucle**, a qualsevol programa de ràdio (emissor o receptor). Escriu-les per al canal **10**.

Solució

```
import radio

radio.config(group=10)
radio.on()
```

Si dues plaques no es «senten», el 90 % de les vegades és una d'aquestes tres línies: falta `radio.on()`, o cada placa és en un `group` diferent. El `group` (0-255) és el «canal privat»: només es reben missatges del mateix grup.

R2 · Emissor de telemetria

Escriu de zero l'emissor complet: canal 10, i cada **2 segons** envia la temperatura llegida amb `temperature()` en forma de text que comenci per `'T: '`.

Solució

```
from microbit import *
import radio

radio.config(group=10)
radio.on()

while True:
    radio.send('T: ' + str(temperature()))
    sleep(2000)
```

`radio.send()` només envia **text**: per això cal `str(...)` al voltant del número. El prefix `'T: '` és protocol: el receptor sabrà que aquest missatge és una temperatura i no una ordre.

R3 · Receptor que actua

Escriu de zero el receptor: canal 10, i a cada volta del bucle comprova si ha arribat el missatge exacte `'F'`; si és així, crida `avancar(400)`.

Solució

```
from microbit import *
import radio

radio.config(group=10)
radio.on()

while True:
    missatge = radio.receive()
    if missatge == 'F':
        avançar(400)
```

Dos punts crítics: `radio.receive()` retorna **None** si no hi ha res (per això es compara amb `==`, que amb **None** simplement dona **False**), i la comparació és `==` (preguntar), **mai** = (assignar).

R4 · Rebre un número sense petar

El receptor rep 'T:23' (text). Escriu el bloc que en **separa** el número amb `missatge[2:]`, el converteix amb `int(...)` i el mostra; si la conversió falla, mostra `Image.SAD` en lloc de petar.

Solució

```
missatge = radio.receive()
if missatge is not None and missatge.startswith('T:'):
    try:
        temp = int(missatge[2:])
        display.scroll(temp)
    except ValueError:
        display.show(Image.SAD)
```

Tot el que arriba per ràdio és **text**, encara que «sembli» un número. La cadena `missatge[2:]` salta el prefix 'T:'. El `try/except` protegeix del missatge corrupte o inesperat: a la prova (i a la vida) no pots suposar que sempre arriba el que esperes.

R5 · Preguntes llampec de protocol

Respon en una frase cadascuna: **(a)** Per què el teu emissor i el del company de la taula del costat no s'interfereixen si cada muntatge usa un `group` diferent? **(b)** Per què convé que els missatges duguin prefix ('T:', 'F'...) en lloc d'enviar números pelats? **(c)** On s'ha d'escriure `radio.on()`: dins o fora del `while True`:? Per què?

Solució

(a) La ràdio només lliura missatges del mateix `group`: grups diferents = canals separats, encara que tothom emeti alhora. **(b)** El prefix identifica **què és** el missatge: el receptor pot distingir una temperatura ('T:') d'una ordre de moviment ('F') i ignorar el que no entén. **(c) Fora** (abans) del bucle: s'activa un sol cop. Dins del bucle no és un error fatal, però és feina inútil repetida mil cops per segon.

Domines les dues targetes sense mirar cap solució? Estàs a punt per a la prova pràctica T3. Si el que falla és la part de sensors o funcions, torna a `00_Repas_expres_MicroPython.md`.